



Agent 时代的 开源协作

这份报告从 143 个全景图项目、100 个高活跃仓库和 5,000 条公开 Issue / PR 出发，观察 Agent 正在怎样进入开源软件。项目全景图正在补齐 Runtime、工具、隔离和可追溯性；在 GitHub 上，Agent 很少发起工作，却已经大量进入评审、讨论、分流和修改代码的环节，而合并、关闭或重新打开线程的最后一个公开动作，绝大多数仍由 GitHub User 账号完成。

研究范围

143 个全景图项目	100 个高活跃仓库
5,000 条公开 Issue 与 PR	8 个月 2026 年 1 月 1 日至 8 月 31 日

目录

01	Agent 时代的开源协作	章节
02	01 · Landscape 与 Open Infrastructure	章节
03	01A · 当前全景图	小节
04	01B · 全景图中的信号	小节
05	01C · 开放基础设施	小节
06	02 · 开源协作	章节
07	02A · 工作量与仓库环境	小节
08	02B · 贡献入口与 Agent 设置	小节
09	02C · Agent 在公开协作中的位置	小节
10	Agent 扩大代码供给，开源协作决定项目能吸收并长期维护多少	小节
11	方法与数据口径	章节
12	References	章节
13	数据、平台与 GitHub	小节
14	Open infrastructure	小节
15	协作治理与研究	小节

这份报告发现了什么

基础设施正在追赶已经形成的应用需求。 Application 占 Agent Infra 7 月 OpenRank 的 55%；相较 5 月 tracking pool 新纳入的 23 个 Agent Infra 项目中，13 个属于 Runtime。新增项目正在补齐上下文、互操作、工具控制和执行环境。	仓库已经开始为 Agent 准备贡献入口，但 Agent 很少发起工作。 Top 100 中有 92 个仓库发布了 coding-agent 文件或目录。在 5,000 条抽样线程中，只有 87 条由具名 Agent 账号或 App 发起，但 2,158 条出现了 Agent。Agent 的公开痕迹主要发生在工作进入仓库之后：评审、讨论、分流和代码修改。	代码供给增长得比合入路径更快。 同一组 55 个仓库中，PR 流入从 2025 年的 129,563 条增加到 2026 年的 265,447 条；90 天后仍保持 open 的比例从 5.5% 升到 11.3%，仓库中位合入率从 77.0% 降到 68.4%。每仓库 PushEvent 账号中位数从 17 个增加到 25 个，公开集成圈在扩大，却没有 PR 队列增长得快。
---	---	---

研究范围

- 2026 年 5 月 tracking pool：227 个仓库；
- 当前 canonical project list：277 个仓库；
- 当前 Agent Infra 与 Model Infra 全景图：143 个项目；
- Agent Infra：84 个；Model Infra：59 个；
- 当前入选但不在 5 月 tracking pool 中：31 个。

这些数字定义了全景图分析的项目范围。协作分析则从其中选取 7 月 OpenRank 最高的 100 个仓库，并进一步构造不同的分析样本。

Landscape 与 Open Infrastructure

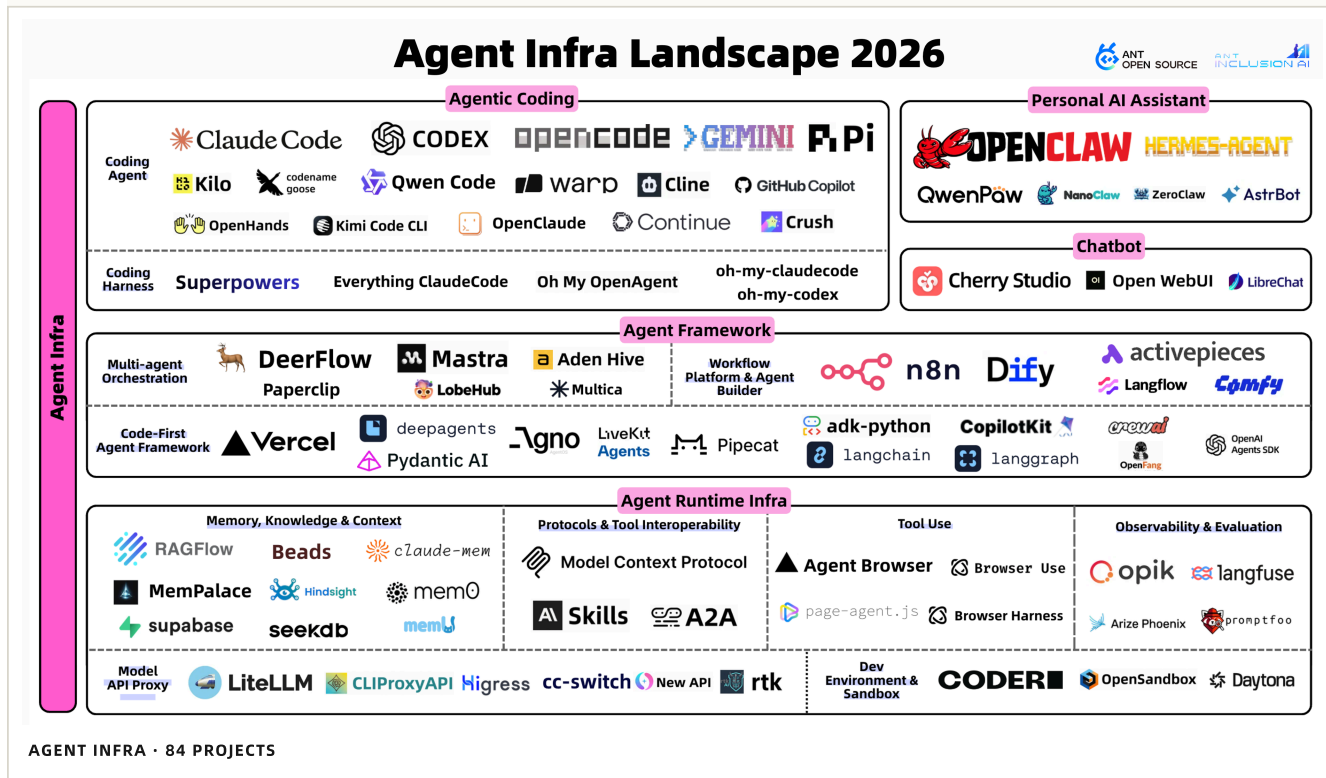
全景图是本研究的起点。它先回答开源工作正在聚集在哪里，再进一步讨论 Agent 对生产基础设施提出了什么要求，以及它如何改变开发协作。Agent Infra 覆盖应用、开发框架和 Agent 完成任务时使用的 Runtime；Model Infra 覆盖模型服务、训练、数据和计算。

01A · 当前全景图

应用承载了最多活跃度，Runtime 正在快速补齐

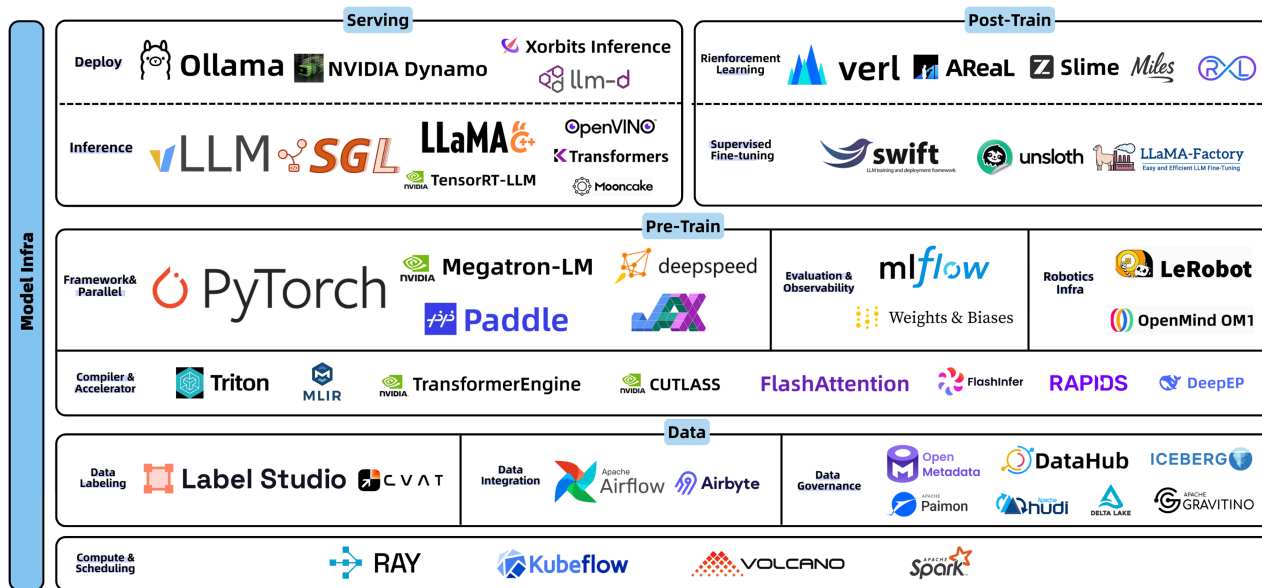
当前全景图包括 84 个 Agent Infra 项目和 59 个 Model Infra 项目。Application 占 Agent Infra 7 月 OpenRank 的 55%，而 Runtime 占 5 月 tracking pool 之外 23 个 Agent Infra 项目中的 13 个。Model Infra 更成熟、以 Python 为主，其中 Serving 占 7 月 OpenRank 的 44%。

Agent Infra 与 Model Infra Landscape 2026



Model Infra Landscape 2026

ANT OPEN SOURCE
INCLUSION AI



MODEL INFRA · 59 PROJECTS

全景图	入选项目	创建于 2025 年或之后	7 月 OpenRank 前五
Agent Infra	84	46 (55%)	OpenClaw · Hermes Agent · Deer Flow · Lark CLI · OpenViking
Model Infra	59	10 (17%)	PyTorch · SGLang · vLLM · Ollama · FlashInfer

交互式报告提供两张完整全景图，可以查看每个仓库的分区、GitHub 元数据和 7 月 OpenRank。这里的入选代表项目通过了本报告对相应生态位置的编辑与活跃度审查，不等同于生产采用证明。

01B · 全景图中的信号

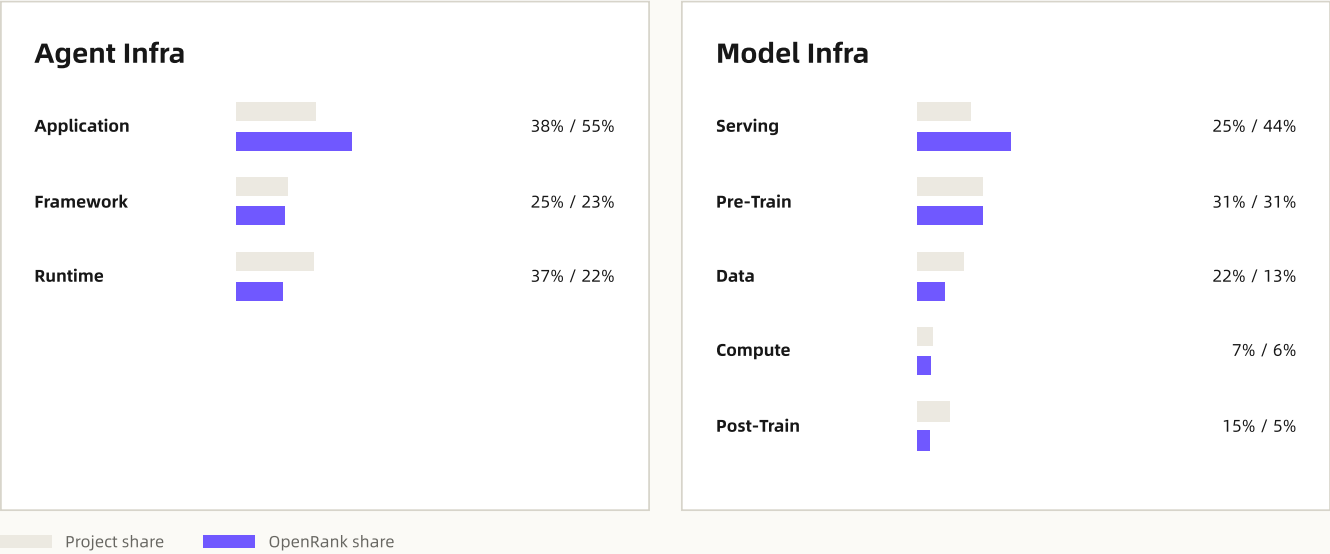
Application 仍然最活跃，新增项目更多出现在 Runtime

自 5 月以来，持续的生态发现把 tracking pool 从 227 个仓库扩展到 277 个。Agent Infra 中，32 个 Application 项目贡献 55% 的 7 月 OpenRank；31 个 Runtime 项目贡献 22%。新增项目则更集中在下层：23 个 5 月池外的 Agent Infra 项目中，Runtime 有 13 个，Application 有 7 个，Framework 有 3 个。

项目先通过活跃度发现和定向 GitHub 搜索进入 tracking pool，再由编辑复核决定是否进入发布版全景图。因此，“不在 5 月池中”表示本次研究覆盖范围有所扩展，既可能是新项目，也可能是后来被纳入研究的老项目。

图 02

各层项目数量与 7 月 OpenRank



Serving 与 Pre-Train 合计占 Model Infra 7 月 OpenRank 的 75%。Model Infra 虽然更成熟，但推理服务仍然承载着最集中的系统工程活动。

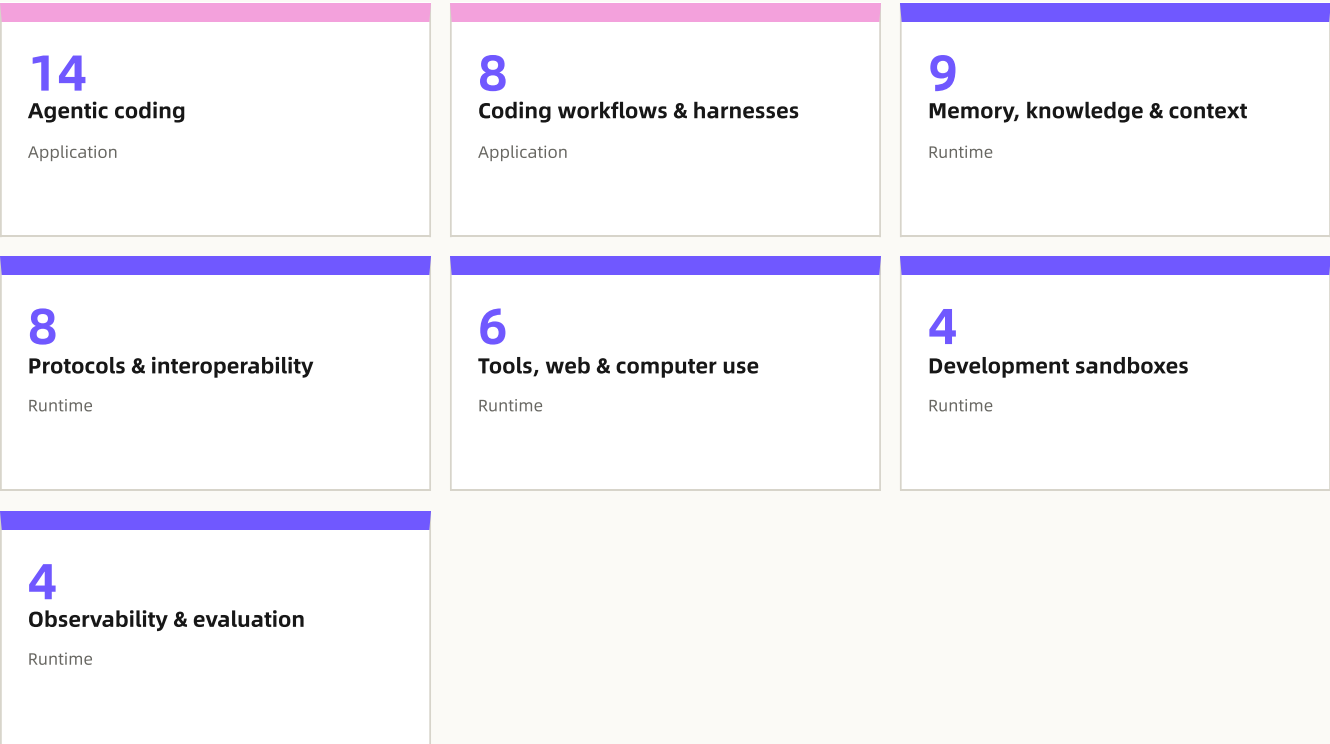
Coding 是委托式软件工作的第一个大规模试验场

当前全景图包含 14 个 Agentic coding 项目和 8 个 coding workflow / harness 项目，是最大的应用集群。代码工作给 Agent 提供了明确的工作区、可执行测试、可见 diff 和评审路径，形成了其他 Agent 场景很少具备的完整反馈回路。

相邻的 Runtime 项目说明 coding 只是入口：9 个项目负责 memory、knowledge 或 context，8 个负责 protocol 与 interoperability，6 个负责 tool、web 或 computer use，4 个提供 development sandbox，另有 4 个负责 observability 与 evaluation。Agent 能修改代码之后，系统还要决定它可以读取什么、调用什么、在哪里运行，以及任务结束后留下什么证据。

图 03

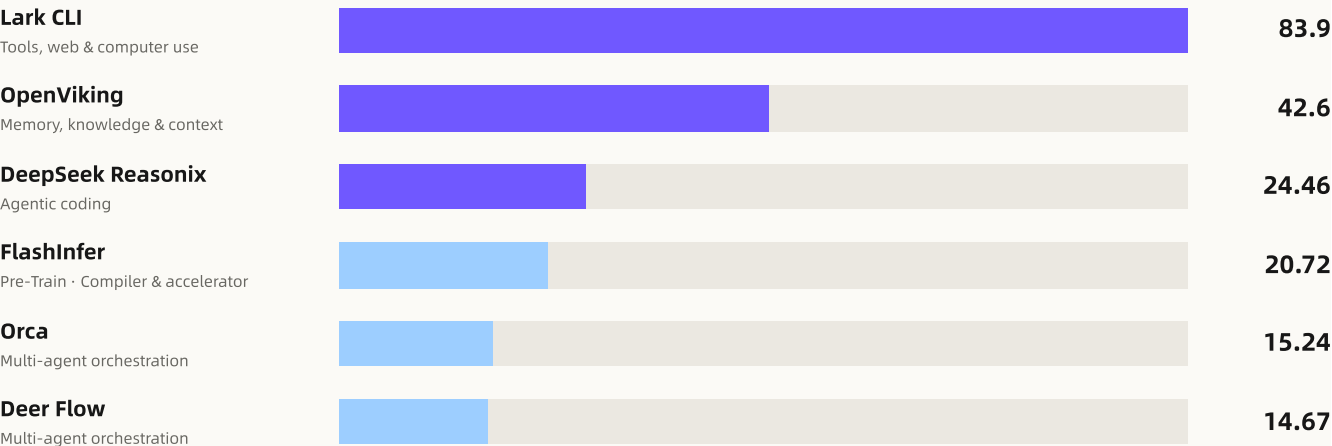
Coding 入口及其相邻 Runtime



活跃度增长正在出现在工具、上下文和推理效率周围

图 04

4 月至 7 月 OpenRank 增长前六

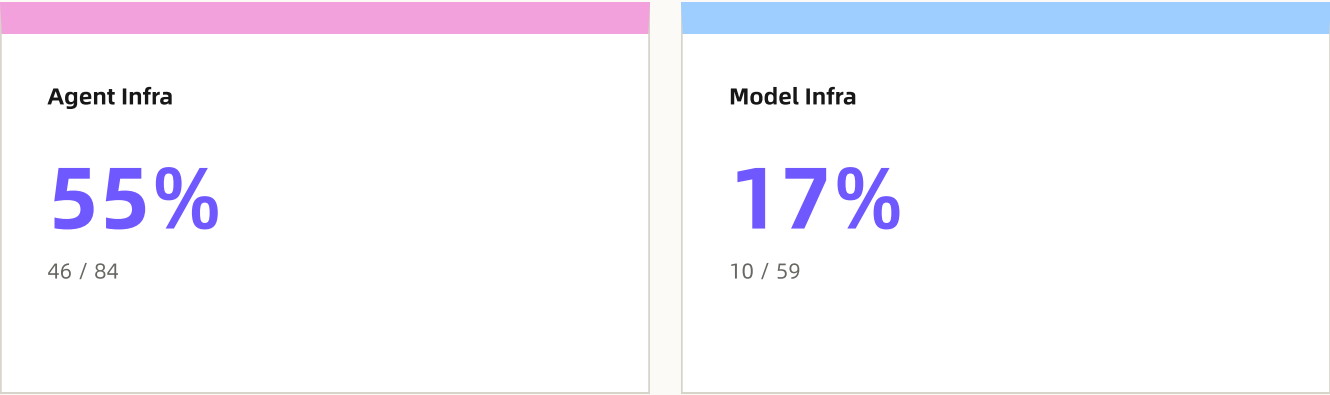


这里比较的是完整月度 OpenRank 的绝对变化。增长最快的项目并不全部来自最拥挤的类别，工具使用、上下文管理和推理效率都出现了明显活动增量。

Agent 层很年轻，承接它的基础设施并不年轻

图 05

入选项目的创建时间

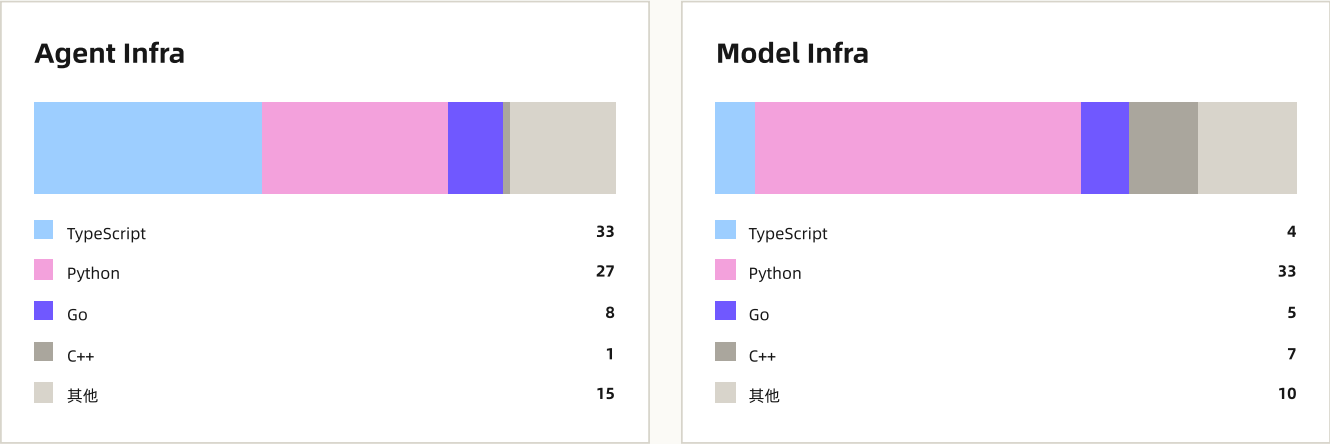


Agent 接口和 Runtime 正在这一轮浪潮中形成；模型服务、训练框架、调度器和数据系统则带着多年工程经验进入 Agent stack。新工作负载要求这些成熟系统处理短生命周期代码、委托式工具访问，以及可能比单个进程存活更久的状态。

Agent 产品更偏 TypeScript，模型基础设施仍以 Python 为主

图 06

GitHub primary language

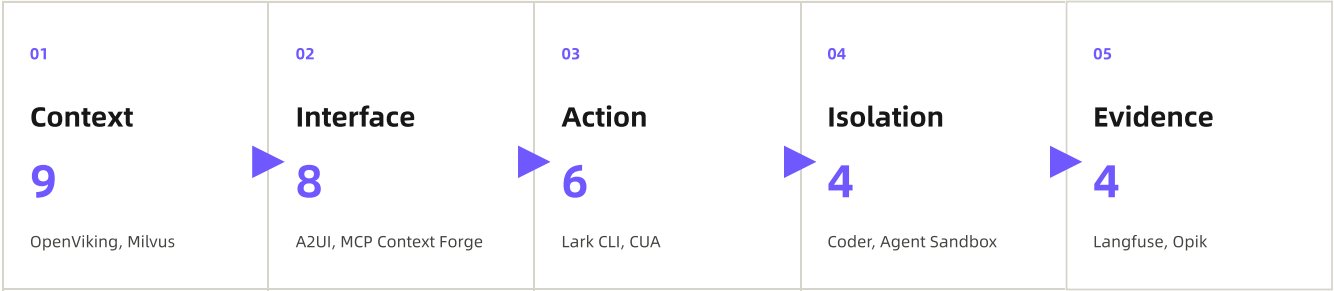


这是 GitHub 的仓库级 primary-language 标签。它便于横向比较，但一个多语言仓库仍只会落在 GitHub 标记的单一主要语言下。

Runtime 项目对应一条完整的 Agent 任务路径

图 07

Agent Runtime 路径



一项 Agent 任务会检索上下文、跨过接口、调用工具、在隔离环境中执行，并留下可供检查的证据。沿着这条路径，应用问题逐步变成基础设施责任：上下文需要生命周期，接口需要策略，工具调用需要有限权限，生成代码需要隔离，外部影响需要留下可追溯记录。

应用问题逐步变成基础设施责任：上下文需要生命周期，接口需要策略，工具调用需要有限权限，生成代码需要隔离，外部影响需要留下可追溯记录。

01C · 开放基础设施

Agent 执行具有波动性、状态性和外部影响

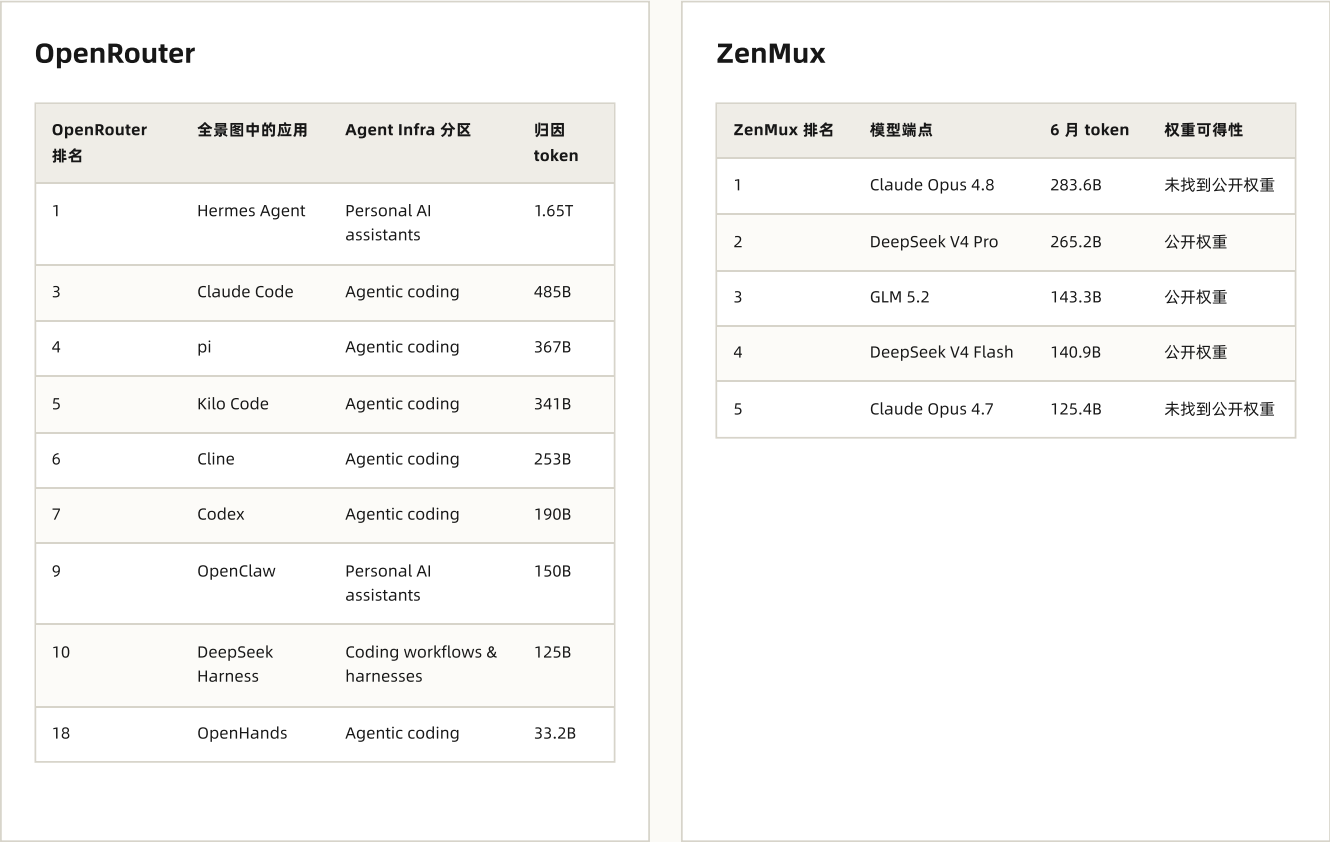
Agent 可以在任务开始后生成代码，扇出为多次模型和工具调用，暂停、重试并改变外部系统。有的任务很短，有的会等待人或远端服务。平台 token 总量能够证明流量存在，却不能给出单个任务的扇出、峰值并发或 QPS。更稳定的基础设施问题是：当组成任务的进程不断出现和消失时，隔离、权限、预算、状态和证据如何始终绑定在同一个任务上。

同一批工作负载也出现在 GitHub 之外

2026 年 8 月 29 日，OpenRouter 公开且自愿归因的全球应用榜 Top 20 中，有 9 个应用能与当前 Agent Infra 全景图直接对应，其中 7 个进入 Top 10。

图 08

GitHub 之外的平台流量



这个对照为仓库全景图增加了一条独立的使用信号：一批在开源仓库中高度活跃的应用，也在公开平台上产生了可见调用量。

OpenRouter 反映 Agent 应用需求，ZenMux 则显示另一个平台上的模型流量中，开放权重模型承担了相当份额。两个平台的 token 不做相加。

ZenMux 提供了另一个单平台视角。其 2026 年 6 月 1-30 日冻结导出中，使用量前四的模型端点里有三个能够链接到官方公开权重仓库：DeepSeek V4 Pro、GLM 5.2 和 DeepSeek V4 Flash。

开放基础设施正在从多个位置承接 Agent 任务

图 09

围绕任务的开放基础设施项目

01 运行与隔离 Kubernetes Agent Sandbox; Kata Containers; Confidential Containers Sandbox 生命周期和 warm pool; VM 级隔离; 面向敏感 AI 工作负载的机密计算基础	02 协调与运行 kagent; Dapr Agents; OpenChoreo Agent 操作 Kubernetes、Prometheus、Istio 和 Argo; durable workflow、状态、重试和 SPIFFE identity; 同一平台以不同接口服务人和 Agent
03 连接与治理 kgateway; agentgateway; Istio LLM、MCP 和 Agent 流量的控制面与数据面; service mesh 和 gateway policy 延伸到 AI 工作负载	04 追踪与解释 OpenTelemetry; Jaeger Agent、workflow 和 execute-tool 语义; 成熟 tracing 项目把 UI 与协议层扩展到 Agent 执行路径

这些项目并不都以 Agent 为核心定位。Confidential Containers 提供与 AI 相关的隔离基础，Istio 正在把成熟服务网格能力延伸到 AI 流量；Prometheus 和 Argo 出现在 kagent 的工具链中，说明 Agent 正在成为既有基础设施的消

费者。

生产 Agent 需要一个 task envelope

本报告用 “task envelope” 指一项任务共同拥有并应当共同结束的 identity、budget、environment、state 和 evidence。

任务需要	已有开放组件	仍然存在的接口缺口
创建短生命周期环境	Agent Sandbox、Kubernetes scheduler、Kueue、DRA	跨 CPU / GPU、模型和数据局部性的联合 placement
隔离不可信代码	Kata Containers、Confidential Containers	根据动作风险选择隔离等级，并保留证明
取得有限权限	SPIFFE / SPIRE、service account、gateway policy	面向单任务、短时、按工具限制的委托
控制模型与工具成本	agentgateway、kgateway、service-mesh telemetry	汇总重试、扇出、模型、工具和 sandbox 的统一预算
暂停、恢复和重试	Dapr Agents、workflow state、checkpoint	带外部副作用任务的幂等恢复和安全取消
改变外部系统	OpenTelemetry、Jaeger	从模型工作、工具执行一直连接到外部结果的完整记录

当前组件已经覆盖了大量基础能力，缺口主要出现在任务级的连接处：预算、取消、安全恢复和证据仍然容易停在组件边界。Agent 工作负载也不是简单的 “更大流量”，而是短时高峰、并行扇出、等待与重试交织，且错误可能已经改变外部世界。Sandbox 因此不再只是启动一个容器，而是需要可创建、claim、预热、回收并保留审计记录的 Runtime 对象。

开源协作

02A · 工作量与仓库环境

研究首先冻结 277 个 tracking pool 中 7 月 OpenRank 最高的 100 个仓库。OpenRank 只用于确定样本，后续判断不再使用它。每个仓库再根据技术角色和与 LLM 的关系进行人工复核。

人工复核得到 68 个 LLM-native、18 个 traditional 和 14 个 mixed 项目。**mixed** 指仓库仍然保有完整的非 LLM 用途，但 AI 或 Agent 已经成为重要产品表面，例如 n8n、Warp 和 MLflow。这个判断不能简单用创建时间代替：LangChain、Megatron-LM 和 TRL 早于 ChatGPT，却本来就围绕语言模型构建；ComfyUI 与 Apache Gravitino 创建得更晚，核心价值却不依赖 LLM。

Top 100 是怎样的一组仓库？

观察维度	分布
技术角色	36 Model Infra · 28 Agent Application · 21 Agent Framework · 15 Agent Runtime Infra
项目身份，人工复核	68 LLM-native · 18 traditional · 14 mixed
仓库创建时间	72 个创建于 2022 年 12 月或之后 · 28 个更早
GitHub primary language	44 Python · 26 TypeScript · 11 Go · 19 其他

这组样本代表 tracking pool 中最活跃的一部分，而不是所有开源项目的普查。仓库规模、发版频率和贡献量都应当在这个范围内理解。

一个仓库框架，两层证据

整章始终从同一份 Top 100 出发，再按数据深度分成两层。

证据层	覆盖范围	回答的问题
仓库全量计数	GitHub Search 找到的 Top 100 在 2024、2025、2026 年 1-8 月全部公开 Issue / PR；历史比较固定为三个成熟窗口里都有活动的 55 个仓库	一共进入多少工作、关闭多少、固定 90 天后还剩多少、队列是否继续增长
仓库平衡的线程时间线	2026 年每仓库 50 条 Issue / PR；历史面板在同一批 55 个仓库中，为 2025 和 2026 各取 2,750 条	opener 之后谁先响应、仓库团队多快出现、PR 经历了几轮可见的评审与修改

工作量与结果主要看全量计数，线程样本用来解释公开协作是怎样发生的。10 条代码沿革 PR 和 7 条易读的公开线程，都从 2026 年线程样本里继续下钻。

PR 正在比 Issue 增长得更快

2026 年 1 月 1 日至 8 月 31 日，Top 100 共开启 349,826 条 Issue 和 606,741 条 PR，平均每条 Issue 对应 1.73 条 PR。月度比值从 1 月的 1.35 上升到完整 8 月的 2.11。

月份	新开 Issue	新开 PR	PR / Issue
1 月	26,320	35,540	1.35×
2 月	35,543	50,329	1.42×
3 月	49,896	74,831	1.50×

月份	新开 Issue	新开 PR	PR / Issue
4 月	50,167	71,173	1.42×
5 月	44,040	78,753	1.79×
6 月	41,604	83,082	2.00×
7 月	49,274	101,482	2.06×
8 月	52,982	111,551	2.11×

历史比较固定为 55 个仓库：它们来自当前 Top 100，并且在 2024、2025、2026 年 1-5 月的固定成熟窗口里都有公开 Issue 或 PR。同一份名单用于队列流量、90 天结果和 PushEvent 核心参与者分析。

同一组 55 个仓库 · 1-8 月	新开 Issue	关闭 Issue	Issue 差额	新开 PR	关闭 PR	PR 差额
2024	53,847	46,033	+7,814	97,793	96,367	+1,426
2025	74,586	64,001	+10,585	129,563	125,104	+4,459
2026	71,539	72,789	-1,250	265,447	222,675	+42,772

“差额”是八个月内新开量减去关闭量；关闭量可以包含更早进入的 backlog。正数表示进入的工作比关闭的更多。2026 年 Issue 基本打平，PR 却翻了一倍，并且比关闭量多出 42,772 条。55 个仓库里有 54 个出现 PR 正增长。问题不是所有协作都处理不动，而是进入 review、修改和合入判断的代码变更增长得更快。

把每条协作项的观察时间固定下来，差距仍然存在。1-5 月创建的 Issue / PR 都从所属月末继续观察 90 天。

同一组 55 个仓库 · 固定 90 天结果	2024	2025	2026
90 天后仍未关闭的 Issue	32.3%	31.7%	28.6%
90 天后仍未解决的 PR	4.6%	5.5%	11.3%
仓库中位 90 天 PR 合入率	81.1%	77.0%	68.4%

Issue 的处理反而略有改善；PR 在同样 90 天里更不容易进入 GitHub 的 merged 状态，仍保持 open 的比例翻了一倍。因此，2026 年的堆积不能只解释成“8 月刚提交，还没来得及处理”。

不同技术角色承受的压力也不一样。

2026 年技术角色	Issue 队列差额	Issue 增长仓库	PR 队列差额	PR 增长仓库	仓库中位 90 天未解决 PR	仓库中位 90 天合入率
Agent Application	+45,865	21 / 28	+54,625	26 / 28	7.0%	56.6%
Agent Framework	+7,407	13 / 21	+17,139	21 / 21	8.7%	68.1%
Agent Runtime Infra	+526	13 / 15	+5,715	15 / 15	6.9%	73.7%
Model Infra	+321	29 / 36	+35,735	36 / 36	11.4%	67.8%

Agent Application 同时积累了问题和代码变更。三类基础设施仓库的 Issue 总量更接近平衡，但每一个 Framework、Runtime 和 Model Infra 仓库都增加了 PR 队列。Model Infra 的仓库中位 90 天未解决 PR 最高，Agent Runtime 的中位合入率最高。

Top 100 的流量本身也高度集中。Claude Code、OpenClaw、Hermes Agent、OpenCode 和 Codex 占了 54.5% 的 Issue；PR 前五名占 34.7%。因此报告同时展示总量、仓库中位数和发生增长的仓库数量，避免把少数超大项目当成全部。

Agent 触达了更多 PR，但 30 天内完成的比例下降了

上面的仓库全量数据说明了队列怎样增长。为了看清队列内部发生了什么，我们比较了同一组 55 个仓库在 2025、2026 年 1-8 月的线程：每个仓库、每年各 50 条，共 5,500 条。“维护者响应”指 opener 之外的仓库 Owner、Member 或 Collaborator 使用 GitHub User 账号作出响应。

同一组 55 个仓库 · 每仓库、每年 50 条线程	2025	2026	变化
出现具名 Agent 或 App 的线程	19.9%	46.1%	+26.3 个百分点
7 天内收到仓库维护者响应	37.1%	31.1%	−6.0 个百分点
PR 在 30 天内处理完成	87.8%	80.3%	−7.5 个百分点
PR 出现公开 review	68.8%	73.9%	+5.1 个百分点

具名 Agent 出现在线程中的比例翻了一倍以上，进入公开 review 的 PR 也更多；与此同时，第一周维护者响应和 30 天 PR 完成率都下降了。Agent 帮助更多变更走到 review，仓库仍需要投入真人注意力并承担最终决定，才能把这些变更处理完。

一部分仓库几乎每天都在发布 GitHub Release

2026 年 1 月 1 日至 8 月 31 日共 243 天。Top 100 中有 98 个仓库至少发布过一次 non-draft GitHub Release。**Release day** 指至少存在一条这类记录的 UTC 日期；中位仓库有 34 个 release day，六个仓库达到 180 天以上。

243 天窗口中的 release days	仓库数
0	2
1	2
2-9	13
10-29	26
30-89	27
90-179	24
180+	6

仓库	Release days	GitHub Release records
ggml-org/llama.cpp	241 / 243	2,041
QwenLM/qwen-code	222 / 243	492
openai/codex	208 / 243	681
router-for-me/CLIPProxyAPI	203 / 243	440
vercel/ai	194 / 243	15,232
flashinfer-ai/flashinfer	185 / 243	221

llama.cpp 在 243 天中的 241 天发布了 GitHub Release。Vercel AI 的 15,232 条记录分布在 194 天，反映 multi-package 与 canary pipeline。这个频率下，release days 更接近自动化交付节奏，而不是偶发的大版本里程碑。仅有 tag、或只发布到 PyPI / npm 等 registry 的版本不在这里。

核心集成圈在扩大，但 PR 增长得更快

PushEvent 可以观察最终写入路由多少账号共同承担。对每个仓库，我们按公开 PushEvent 次数排序，计算覆盖一半 push 需要多少账号。同一组 55 个 Agentic AI 仓库中，发生过 PushEvent 的账号中位数从 2024 年的 13 个增加到 2025 年的 17 个、2026 年的 25 个；完成一半 push 所需账号从 2 个增加到 3 个。

同一组 55 个 Agentic AI 仓库	PushEvent 账号中位数	完成一半 push 的账号数	前五账号完成的 push 占比中位数
2024	13	2	91.4%
2025	17	2	86.9%
2026	25	3	74.1%

核心圈没有收缩。更多账号进入公开写入路径，push 对前五账号的集中度也在下降。但 2025 到 2026 年，PushEvent 账号中位数增长 47%，同一批仓库的 PR 流入增长 105%。

我们再用 OpenDigger 的技术标签取出活跃的云原生和大数据仓库作对照。2026 年，Agentic AI 仓库完成一半 push 需要的账号中位数为 3，云原生为 2，大数据为 1。

2026 年对照	活跃仓库	PushEvent 账号中位数	完成一半 push 的账号数
Agentic AI 固定面板	55	25	3
云原生	98	12	2
大数据	56	7	1

PushEvent 里的账号是执行仓库写入的 pusher，不一定等于 commit author。它适合衡量有多少公开账号共同承担集成路径。Agentic AI 的核心圈比两组传统技术对照更宽，因此当前压力不能简单解释成“核心参与者越来越少”；真正的差距在增长速度上。

02B · 贡献入口与 Agent 设置

大多数仓库允许外部账号创建 PR

图 10

Top 100 当前协作表面

100 / 100 Issues 已启用	100 / 100 Pull Requests 已启用	98 / 100 任何人可以创建 PR	2 / 100 仅 collaborators 可以创建 PR
74 / 100 Discussions 已启用	89 / 100 找到 CONTRIBUTING	95 / 100 找到 Issue template	84 / 100 找到 PR template

API 设置、贡献文档和实际结果描述的是三个不同层面：能不能打开 PR、项目希望贡献者怎样参与，以及最终哪些变化被接受。人工检查发现，48 个仓库明确邀请外部贡献，12 个要求先开 Issue、事先取得同意或限定贡献范围，38 个未发现限制性信号；Codex 和 Claude Code 两个仓库保留可见的 Pull Requests，但只允许 collaborators 创建。

Mastra 要求代码贡献者先开 Issue。Open WebUI 对首次贡献者设置相同门槛，但本地化工作除外。DeepSeek Harness 不在 Top 100 分母内，它公开 MIT 代码并开放 Discussions，却关闭核心仓库的 Issues 与 PR，把外部开发引向插件。

Coding-agent 设置已经遍布整个技术栈

在 Top 100 的默认分支上，有 92 个仓库存在专为 coding-agent workflow 创建的内容，包括 AGENTS.md、CLAUDE.md 等 instruction file，以及 .claude、.cursor、.codex、.gemini 等工具目录。报告把两类内容合并为“仓库已经为 coding agent 做过设置”，并排除只残留在 .gitignore 中的名字。

默认分支上的文件	仓库
多种 coding agent 可共用的 instructions	80 / 100
Claude Code	71 / 100
Codex	22 / 100
GitHub Copilot	20 / 100
Cursor	17 / 100
Gemini	12 / 100

一个仓库可以同时支持多个工具。LobeHub、Opik、Cline 和 OmniRoute 都发布了四种 agent-specific 格式。这个数字衡量维护者已经提交到默认分支的兼容工作，不代表各种工具的实际使用频率相同。

这些规则已经进入 Model Infra

图 11

不同技术类型的 coding-agent 设置



PyTorch、Spark、Iceberg、ONNX Runtime、Milvus、Triton 和 OpenVINO 都已经出现这类文件。Agent instructions 不只告诉工具怎样写代码：在 86 个可以读取明确 instruction 的仓库里，81 个提到测试或验证，79

个提到 Issue 或计划，72 个提到 code review，63 个提到 release 或 dependency 工作。维护者正在把整个贡献循环写成 Agent 可以读取的规则。

02C · Agent 在公开协作中的位置

5,000 条线程样本追踪公开协作细节

每个仓库抽取 50 条互不重复的线程，最终得到 1,433 条 Issue 和 3,567 条 PR，共 5,000 条。它们来自八个月窗口中的 956,567 条 Issue / PR。每条线程只计算一次，不按仓库总流量加权。

Agent 主要在贡献到达之后进入

具名 coding 或 review Agent 在 2,158 / 5,000 条线程中留下了公开动作，覆盖 95 / 100 个仓库；只有 87 条线程由 Agent 账号或 App 发起。也就是说，样本中 43.16% 出现 Agent，而 Agent 发起只占 1.74%。常见形式是 CodeRabbit 评审 PR、Gemini Code Assist 留下 review comment，或 OpenHands 通过 GitHub App 行动。

本报告只在 GitHub 明确显示 Agent、显示 App，或贡献记录明确声明由 Agent 生成时进行归因。Dependabot、GitHub Actions、release bot 等传统自动化另行分类。开发者在本地使用 Cursor、Claude Code 或 Codex，通常仍显示为普通 User 账号；没有额外公开痕迹时，不根据文风、时间或代码风格猜测。

图 12A

一条公开 Issue / PR 的不同阶段出现了谁



GitHub User 账号 是 GitHub 报告 actor type 为 User 的账号；仓库团队账号 是 GitHub 将其与仓库关联为 OWNER、MEMBER 或 COLLABORATOR。列之间可以重叠，因为团队账号通常也是 User，App 也可能中介 User 的动作。

Agent 产生的公开事件进一步说明它们在做什么：

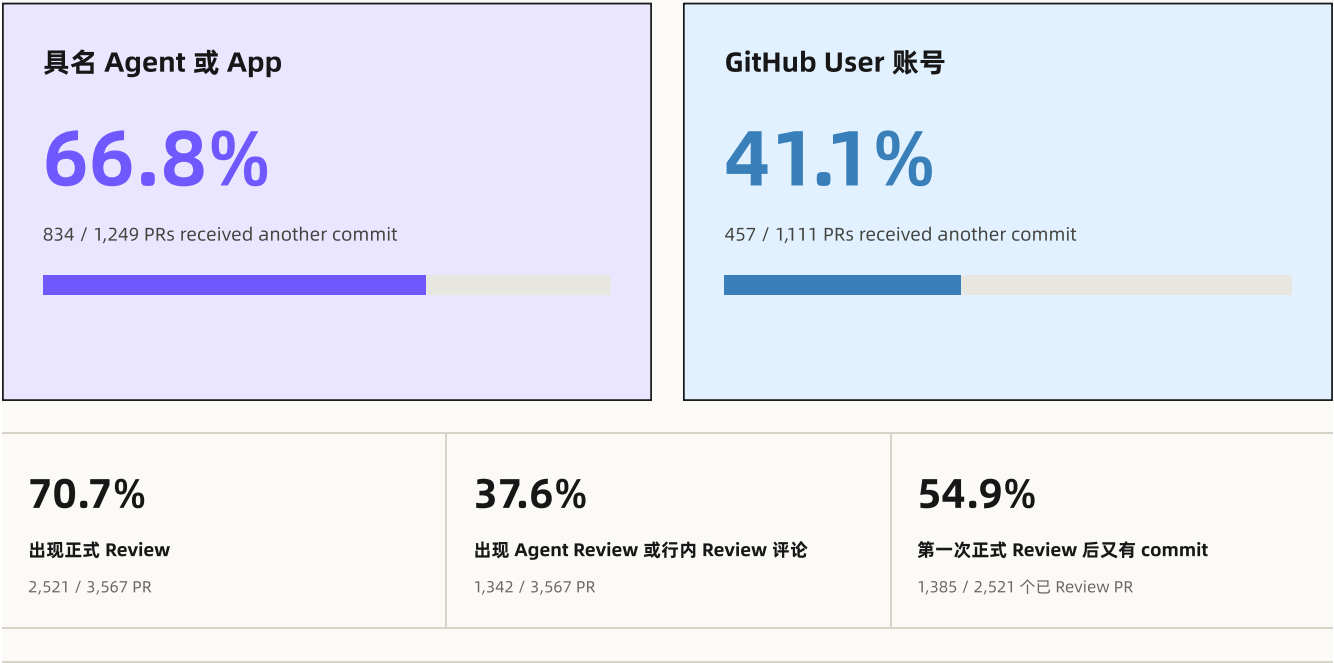
这里的单位是事件，因此一条线程可能贡献多次 review 或 reply。它不能换算为劳动份额，但能够直接显示具名 Agent 服务集中在工作流的哪些位置。

第一次正式 Review 来自谁，后续修改差异很大

我们按时间排列 3,567 条抽样 PR 的正式 Review 和 commit。2,521 条出现过正式 Review，1,342 条出现过 Agent Review 或行内 Review 评论；在全部已 Review 的 PR 中，1,385 条在第一次 Review 后又提交了代码。

图 12B

按第一次正式 reviewer 比较后续 commit



第一次正式 Review 来自具名 Agent 或 App 时，1,249 条 PR 中有 834 条继续提交代码；第一次正式 Review 来自 GitHub User 账号时，1,111 条中有 457 条继续提交。至少在这批公开记录中，Agent Review 并没有停在评论区，它后面更常接着出现一轮代码修改。

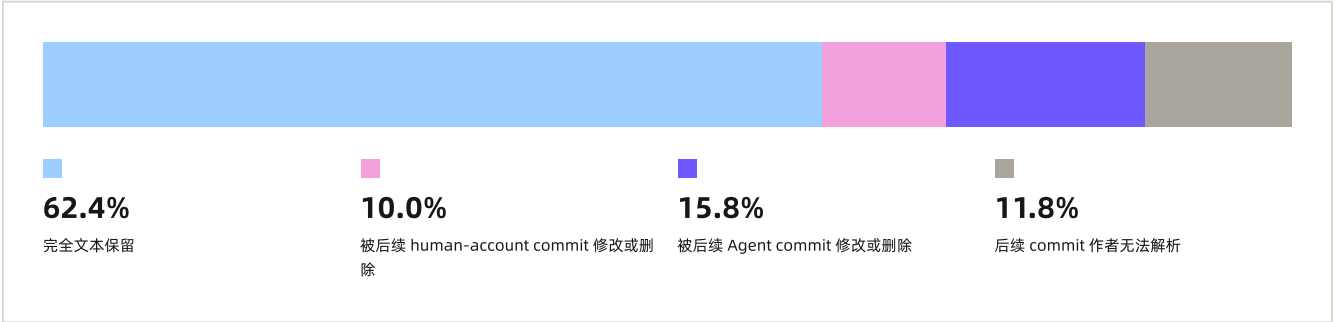
第一版 Agent patch 常常会保留，也可能被人或另一个 Agent 重写

我们追踪了 10 个已合并、且由高置信 coding Agent 发起贡献或编写 commit 的 PR。其中 9 个可以从第一版有效 Agent patch 一直追到 PR 最终 head；Mooncake #2686 的 Agent commit 是 two-parent merge，first-parent diff 混入大量上游行，因此保留在案例册中，但不计入行级分母。

9 个可追踪 PR 的第一版 Agent commit 共新增 1,225 行文本；最终 head 中 765 行（62.4%）仍是完全相同的文本，123 行先被后续 human-account commit 修改或删除，193 行被后续 Agent commit 修改，144 行的后续作者无法解析。

图 12B.1

第一版可观察 Agent patch 后来发生了什么



Pull request	第一版 Agent patch	完全保留	人类账号修改	后续 Agent 修改	作者不明	路径
vercel/ai #18818	172	0	0	172	0	Agent 继续迭代直至合并
warpcotdev/warp #13382	44	31	12	1	0	Agent → human
OpenMetadata #25243	62	21	29	12	0	Agent → human
ONNX Runtime #28045	611	533	78	0	0	Agent → human
OpenHands #2614	11	0	4	7	0	Agent → human
MLflow #19721	262	118	0	0	144	Agent → 未解析作者
MLflow #21621	33	33	0	0	0	Agent 继续迭代直至合并
MLflow #22355	25	25	0	0	0	Agent → human
MLflow #22659	5	4	0	1	0	Agent 继续迭代直至合并

这组行级记录揭示了合并结果背后的不同交接。在 ONNX Runtime #28045 中，611 行第一版 patch 有 533 行保留，78 行被后续 User 账号修改；Vercel AI #18818 的 172 行则全部被后续 Agent commit 替换。仅看最终 merged diff，会错过补丁究竟被保留、由人修改，还是由另一个 Agent 重写。

统计单位是第一条可归因 Agent commit 中新增的精确文本行，空的 planning commit 被跳过。每一行沿后续 commit 继续追踪，第一个修改或删除它的 commit 获得公开账号归因。这是文本级的可复现比较，不等于语义作者权，也看不到普通 User 账号背后的未披露 AI 使用。

DeepSeek Harness 选择了不同的开放治理方式

DeepSeek Harness 不在 Top 100 中，但提供了一个有价值的对照。仓库于 2026 年 8 月 13 日公开，至 8 月 30 日达到 204,176 Stars 和 23,597 forks。代码采用 MIT，Discussions 开放；Issues 关闭，两次检查中 Pulls endpoint 返回 404，贡献指南把社区开发引向第三方插件，同时暂不开放核心 PR。

图 12C

DeepSeek Harness 的贡献表面

源代码	Issues	Pull requests	Discussions	扩展路径
公开 · MIT	关闭	关闭	开放	Plugins

代码可读、核心代码可贡献、以及存在开放扩展生态，是三个不同的治理选择。

7 条公开线程展示了具体交接

交互式案例册包括 [Coder #25800](#)、[ONNX Runtime #28045](#)、[LangChain #37607](#)、[PyTorch #182986](#)、[Supabase #42193](#)、[Gemini CLI #24026](#) 和 [n8n #33411](#)。每条线程都能清楚看到谁提出工作、Agent 在哪里进入、谁修改，以及谁最终结束公开流程。

外部账号提供大多数 PR，User 账号仍然执行最终公开动作

外部账号创建了抽样 PR 的 66.8%。在固定成熟度检查点，已解决 external PR 中 57.2% 显示为 merged；maintainer / member PR 为 82.8%。代码供给可以来自广泛外部参与者，但仓库权限、上下文和持续关系仍然影响变更怎样通过 gate。

GitHub User 账号在 60.1% 的抽样线程中于 opener 之后响应，维护者关联账号为 38.7%。27.6% 的线程在 opener 之后只看到自动化响应；整条线程完全没有 User 账号的情况只有 0.56%。自动化承担了更多分流和响应工作，但例外处理和仓库状态改变仍然集中在 User 账号上。

Agent 扩大代码供给，开源协作决定项目能吸收并长期维护多少

公开记录显示，一种新的分工已经出现。越来越多仓库发布 Agent 可以直接读取的规则；Agent 在绝大多数样本仓库中参与 review、triage、discussion 和 revision；外部账号继续提供大多数 PR；而 merge、close 或 reopen 的最后一个公开动作，仍然主要由 GitHub User 账号完成。

同期证据把这项变化带来的压力展示得更清楚：PR 供给翻倍，进入 review 的比例上升，但及时的人类响应和 PR 完成率下降；Issue 流量则基本平衡。Agent 正在增加生产与 review 产能，却还没有消除判断、集成和长期维护上的约束。行级案例说明了其中的原因：第一版 Agent patch 可能原样保留、被人修改，也可能在成为项目愿意承担的代码之前，被另一个 Agent 完全替换。

下一项值得跟踪的指标从仓库 gate 之后开始：已接受的变化是否被 revert、是否需要 follow-up fix、贡献者是否再次参与，以及它能否继续经受测试和 benchmark。

在 Agent 时代，patch 只是一次贡献的开头。贡献还意味着理解项目认可的问题、遵守它的规则、回应评审，并留下其他人愿意继续维护的代码。**Agent 可以扩大代码供给；开源协作决定一个项目能够负责地吸收多少，并把多少变化继续带向未来。**

方法与数据口径

- 当前项目列表来自 `data/agentic-ai-projects.csv`；5 月基线来自 `data/history_snapshot/2605_agentic_projects.csv`。
- OpenRank 使用完整的 2026 年 7 月；Stars 和 GitHub primary language 使用 8 月 23 日更新的 canonical snapshot。
- OpenRouter App & Agent ranking 公开且为 opt-in；Top 20 对照检查于 2026 年 8 月 29 日。
- ZenMux 数字来自 2026 年 6 月 1-30 日冻结的单平台导出；“公开权重”表示找到官方 public-weight repository，不是 OSI license 判断；不与 OpenRouter 流量相加。
- 开放基础设施部分引用项目文档证明相应工程能力正在出现，不把项目合作、文档或社区活动写成生产部署范围。
- 协作研究冻结 Top 100，并从每个仓库抽取 50 条 2026 年 1-8 月互不重复的 Issue / PR，共 5,000 条。每条线程计算一次，不按仓库流量重新加权。
- 历史流量和 90 天结果使用同一组 55 个仓库的 GitHub 全量计数；响应与修改过程使用这 55 个仓库在 2025、2026 年各 2,750 条同期样本。
- PushEvent 集中度来自 OpenDigger ClickHouse 事件数据；云原生和大数据对照使用 OpenDigger 技术标签，以及 2026 年 7 月有 OpenRank 的活跃仓库。
- 公开 actor 标签区分已验证 Agent 服务、传统自动化、App-mediated User action 和 GitHub User account。普通 User 账号背后的未披露本地 AI 使用不可见。
- Review-to-commit 顺序使用专门的 PR commit timestamp；没有 timestamp 的 timeline commit row 不被解释为“没有后续修改”。
- GitHub 的 merged flag 是可观察 gate 信号，不等同于普适的贡献质量判断。
- GitHub 关注度、公开协作和生产采用在报告中始终是不同的概念。
- DeepSeek Harness 的仓库设置和贡献指南需要在正式发布日再次检查。

详细研究设计： `../research/open-collaboration-study-design.md` Landscape 数据与图表映射： `../research/landscape-signals.md`
Open infrastructure 证据： `../research/open-infrastructure-trends.md`

References

数据、平台与 GitHub

- [GitHub REST API documentation](#)
- [GitHub GraphQL pull request types](#)
- [GitHub timeline events API](#)
- [GitHub Issue 与 Pull Request 搜索条件](#)
- [OpenRank metric documentation](#)
- [OpenDigger 数据说明](#)
- [OpenDigger labeled data](#)
- [OpenRouter App & Agent Rankings](#)
- [ZenMux App Leaderboard API](#)
- [ZenMux Model Leaderboard API](#)

Open infrastructure

- [Agent Sandbox quickstart](#)
- [Agent Sandbox threat model](#)
- [Kubernetes Agent Sandbox roadmap](#)
- [Kata Containers Agent Sandbox integration](#)
- [OpenInfra Foundation projects](#)
- [Deploying the SPIRE Agent](#)
- [OpenTelemetry GenAI agent span conventions](#)
- [Kagent: Bringing Agentic AI to Cloud Native](#)
- [Dapr Agents v1.0](#)
- [OpenChoreo and the agentic enterprise](#)
- [Kgateway v2.1 and agentgateway](#)
- [Agentgateway request and token rate limits](#)
- [Agentgateway MCP per-tool rate limits](#)
- [Istio in the AI era](#)
- [Jaeger tracing AI agents with OpenTelemetry](#)
- [Confidential Containers becomes a CNCF incubating project](#)
- [Kubernetes v1.34 Dynamic Resource Allocation updates](#)
- [Kueue overview](#)
- [Kueue topology-aware scheduling](#)

协作治理与研究

- [DeepSeek Harness repository](#)
- [DeepSeek Harness contribution guide](#)
- [Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity](#)
- [We are Changing our Developer Productivity Experiment Design](#)

- [AIDev: Studying AI Coding Agents on GitHub](#)
- [AIDev open dataset](#)
- [On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub](#)
- [Where Do AI Coding Agents Fail?](#)
- [From Industry Claims to Empirical Reality: An Empirical Study of Code Review Agents in Pull Requests](#)
- [Security in the Age of AI Teammates](#)
- [Understanding the Rejection of Fixes Generated by Agentic Pull Requests](#)
- [AI Agent Pull Requests on GitHub: Frequency, Structure, and Merge Conflict Rates](#)
- [State of Open Source AI](#)